Lekcja 6: Programowanie strukturalne w Pythonie

`#lekcja` `#python` `#funkcje` `#struktury` `#zakresy`

W tej lekcji wchodzimy na wyższy poziom organizacji kodu. Zrozumiemy, dlaczego dzielenie programu na mniejsze, logiczne części jest kluczowe. Nauczymy się tworzyć własne funkcje, zarządzać widocznością zmiennych, a także pisać kod, który jest nie tylko działający, ale również czytelny i dobrze udokumentowany dla innych (i dla nas samych w przyszłości).

1. Czym jest programowanie strukturalne?

> **Definition**
>
> Programowanie strukturalne to paradygmat programowania, który opiera się na podziale dużego problemu na mniejsze, zarządzalne bloki. Zamiast pisać jeden długi ciąg poleceń (tzw. "spaghetti code"), używamy trzech podstawowych konstrukcji:
>
> 1. **Sekwencja:** Wykonywanie instrukcji jedna po drugiej.
> 2. **Wybór:** Używanie instrukcji warunkowych (`if` , `elif` , `else`) do podejmowania decyzji.
> 3. **Iteracja:** Używanie pętli (`for` , `while`) do powtarzania zadań.

Głównym narzędziem do osiągnięcia tego celu w Pythonie są **funkcje**.

Zalety programowania strukturalnego:

- **Czytelność:** Kod jest łatwiejszy do zrozumienia.
- **Modularność:** Program składa się z niezależnych "klocków" (funkcji).
- **Łatwiejsze testowanie i debugowanie:** Można testować każdą funkcję osobno.
- **Wielokrotne użycie kodu (DRY - Don't Repeat Yourself):** Zamiast kopiować ten sam kod, wywołujemy funkcję.

2. Funkcje i argumenty

Funkcja – co to takiego?

Funkcja to nazwany blok kodu, który wykonuje określone zadanie. Można ją wywoływać wielokrotnie w różnych miejscach programu.

Anatomia funkcji:

```python
# Słowo kluczowe 'def', nazwa funkcji i parametry w nawiasach
def nazwa_funkcji(parametr1, parametr2):
    # Ciało funkcji (kod do wykonania)
    wynik = parametr1 + parametr2
    # Słowo kluczowe 'return' zwraca wartość z funkcji
    return wynik
```

Funkcja z wartością zwracaną i bez niej

- Funkcja, która używa `return`, **zwraca wartość**, którą można przypisać do zmiennej.
- Funkcja bez instrukcji `return` (lub z `return` bez wartości) automatycznie zwraca specjalną wartość `None`.

```python
def dodaj(a, b):
    return a + b

def przywitaj_sie(imie):
    print(f"Cześć, {imie}!")

wynik_dodawania = dodaj(5, 3)
wynik_powitania = przywitaj_sie("Anna")

print(f"Wynik z funkcji dodaj: {wynik_dodawania}")   # Wynik z funkcji
dodaj: 8
print(f"Wynik z funkcji przywitaj_sie: {wynik_powitania}") # Wynik z funkcji
przywitaj_sie: None
```

Rodzaje argumentów

Argumenty pozycyjne i nazwane (kluczowe)

- **Pozycyjne:** Wartości są przypisywane do parametrów na podstawie ich kolejności.
- **Nazwane (kluczowe):** Wartości są przypisywane do parametrów na podstawie ich nazwy.

```python
def przedstaw(imie, wiek, miasto):
    print(f"{imie} ma {wiek} lat i mieszka w mieście {miasto}.")

# Wywołanie pozycyjne
przedstaw("Jan", 30, "Warszawa")

# Wywołanie z argumentami nazwanymi (kolejność nie ma znaczenia)
przedstaw(wiek=40, miasto="Kraków", imie="Kasia")
```

```
# Mieszane (pozycyjne muszą być przed nazwanymi)
przedstaw("Piotr", miasto="Gdańsk", wiek=25)
```

Argumenty domyślne

Parametry mogą mieć wartości domyślne, które są używane, jeśli argument nie zostanie podany przy wywołaniu.

```
def stworz_uzytkownika(imie, jest_aktywny=True, punkty=0):
    print(f"Użytkownik: {imie}, Aktywny: {jest_aktywny}, Punkty: {punkty}")

stworz_uzytkownika("Tomek") # Użyje wartości domyślnych
stworz_uzytkownika("Zofia", jest_aktywny=False)
stworz_uzytkownika("Adam", punkty=100)
```

Dowolna liczba argumentów: `*args` i `kwargs`**

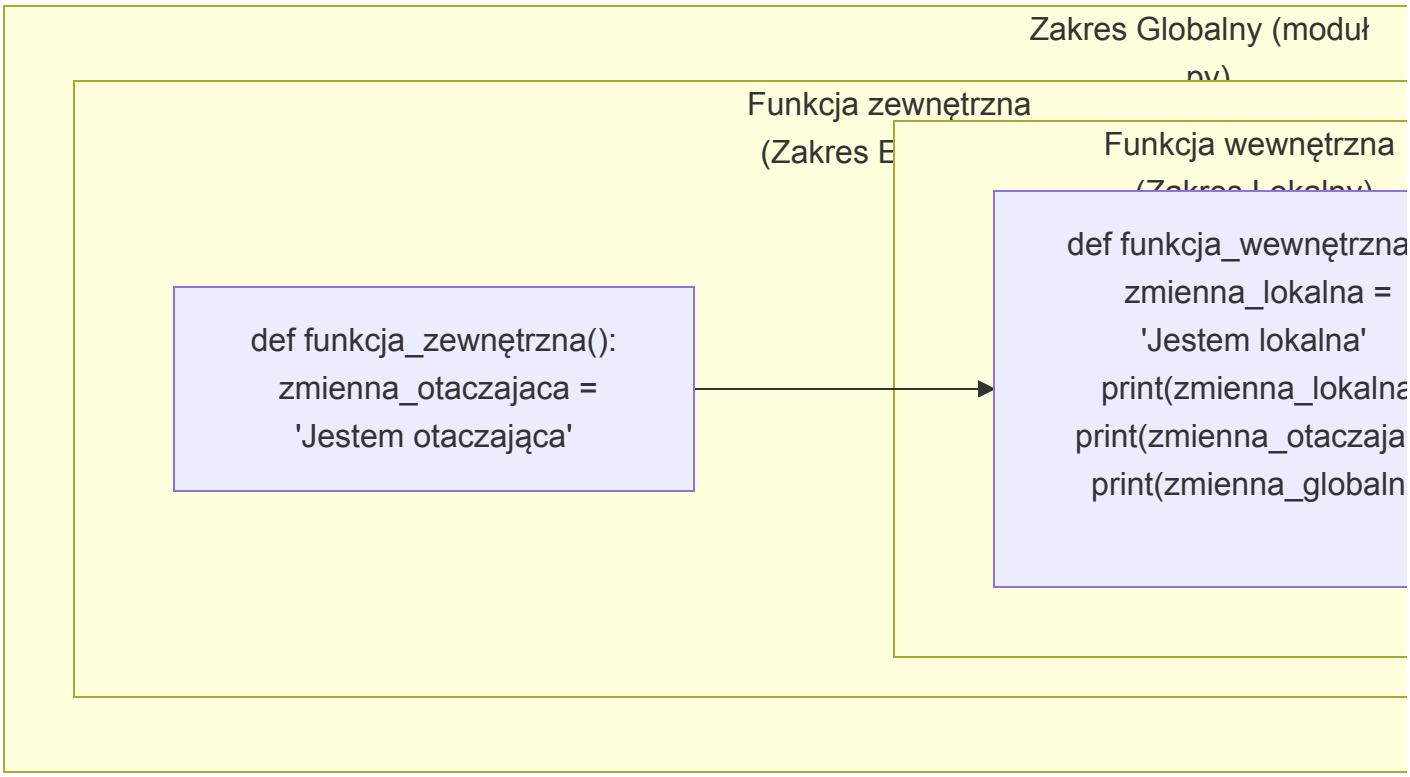
- `*args` : Zbiera wszystkie dodatkowe argumenty **pozycyjne** w krotkę (tuple).
- `**kwargs` : Zbiera wszystkie dodatkowe argumenty **nazwane** w słownik (dict).

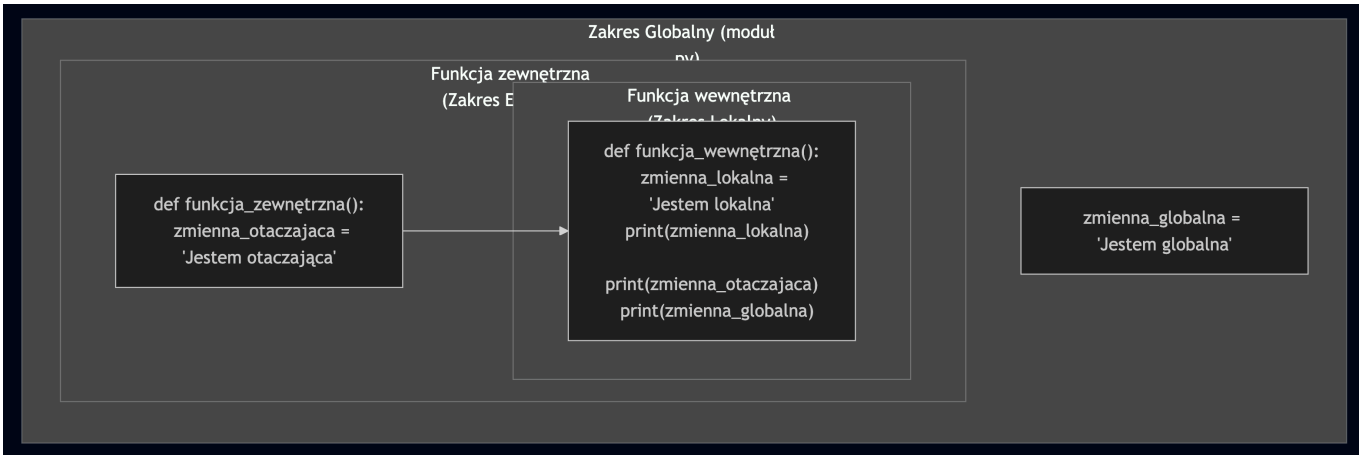
```
def pokaz_wszystko(*args, **kwargs):
    print("Argumenty pozycyjne (args):", args)
    print("Argumenty nazwane (kwargs):", kwargs)

pokaz_wszystko(1, "test", 3.14, imie="Jan", wiek=50)
# Argumenty pozycyjne (args): (1, 'test', 3.14)
# Argumenty nazwane (kwargs): {'imie': 'Jan', 'wiek': 50}
```

3. Zakresy zmiennych (Scope)

Zakres zmiennej (ang. *scope*) decyduje o tym, gdzie w kodzie dana zmienna jest widoczna i dostępna. Python stosuje regułę **LEGB**:

1. **L (Local)** — Zakres lokalny, wewnątrz funkcji.
2. **E (Enclosing)** — Zakres otaczający, w funkcjach zagnieżdżonych.
3. **G (Global)** — Zakres globalny, na najwyższym poziomie modułu.
4. **B (Built-in)** — Zakres wbudowany, funkcje takie jak `print()` , `len()` .





`global` vs `nonlocal`

- `global`: Używane wewnątrz funkcji, aby zasygnalizować, że chcemy zmodyfikować zmienną z zakresu globalnego.
- `nonlocal`: Używane w funkcji zagnieżdżonej, aby zasygnalizować, że chcemy zmodyfikować zmienną z zakresu otaczającego (ale nie globalnego).

```
x = "globalny"

def funkcja_zewnetrzna():
    y = "otaczający"
    def funkcja_wewnetrzna():
        global x
        nonlocal y
```

```
        x = "zmieniony globalny"
        y = "zmieniony otaczający"
    funkcja_wewnetrzna()
    print("Wartość y po zmianie:", y)

funkcja_zewnetrzna()
print("Wartość x po zmianie:", x)
```

> ⚠ **Warning**
>
> Unikaj używania słowa kluczowego global! Modyfikowanie zmiennych globalnych z wnętrza funkcji prowadzi do trudnego do śledzenia kodu ("spaghetti"). Lepszym podejściem jest przekazywanie wartości jako argumenty i zwracanie wyników.

4. Adnotacje typów (Type Hinting)

Od wersji 3.5 Python pozwala dodawać **adnotacje typów**. Są to wskazówki dotyczące oczekiwanych typów danych dla zmiennych, parametrów funkcji i wartości zwracanych.

Korzyści:

- **Czytelność:** Kod staje się formą dokumentacji.
- **Wsparcie dla narzędzi:** Edytory kodu (jak VSCode) używają ich do lepszego autouzupełniania i wykrywania błędów.
- **Analiza statyczna:** Narzędzia takie jak `Mypy` mogą sprawdzać poprawność typów jeszcze przed uruchomieniem programu.

```
# Adnotacje w funkcji
def powitanie(imie: str, ilosc: int = 1) -> str:
    return f"Cześć, {imie}! " * ilosc

# Adnotacje dla zmiennych
wiek: int = 30
lista_imion: list[str] = ["Anna", "Piotr"]
```

> 🖊 **Note**
>
> Adnotacje to tylko wskazówki. Interpreter Pythona ignoruje je podczas wykonywania kodu. Nie zapobiegają one przypisaniu wartości innego typu do zmiennej.

5. Dokumentowanie kodu

Komentarze (`#`)

Komentarze jednoliniowe służą do wyjaśniania **"jak"** lub **"dlaczego"** działa dany fragment kodu. Odpowiadają na pytania, które mogą się pojawić podczas czytania skomplikowanej logiki.

```
# Obliczamy podatek VAT (23%)
podatek = cena * 0.23
```

Docstringi (`"""...""""`)

Docstringi (łańcuchy dokumentujące) służą do opisywania **"co"** robi dana funkcja, klasa lub moduł. To oficjalna dokumentacja, która powinna znaleźć się zaraz po definicji obiektu.

Dobry docstring zawiera:

- Krótkie, jednowierszowe podsumowanie.
- Dłuższy, opcjonalny opis.
- Opis argumentów (`:param`).
- Opis zwracanej wartości (`:return:`).

```
from typing import Union

def oblicz_cene_brutto(netto: Union[int, float], stawka_vat: float = 0.23) -
> float:
    """Oblicza cenę brutto na podstawie kwoty netto i stawki VAT.

    Funkcja przyjmuje kwotę netto oraz opcjonalną stawkę VAT,
    a następnie zwraca pełną kwotę brutto.

    :param netto: Kwota netto (liczba całkowita lub zmiennoprzecinkowa).
    :param stawka_vat: Stawka podatku VAT (domyślnie 0.23).
    :return: Obliczona kwota brutto.
    """
    return netto * (1 + stawka_vat)
```

Tip

Możesz wyświetlić docstring funkcji za pomocą help(nazwa_funkcji) lub print(nazwa_funkcji.**doc**).

🧪 6. Zadania do samodzielnej pracy

1. **Kalkulator:** Napisz funkcję `kalkulator(a, b, operacja)`, która przyjmuje dwie liczby i string z operacją (`"+"`, `"-"`, `"*"` lub `/`"). Funkcja powinna zwracać wynik odpowiedniego działania.
2. **Informacje o książce:** Stwórz funkcję `opis_ksiazki(tytul, autor, rok_wydania=2024)`. Funkcja powinna zwracać sformatowany string, np. `"Książka '[Tytuł]' została napisana przez [Autor] i wydana w roku [Rok wydania]."`. Przetestuj ją, wywołując z argumentami pozycyjnymi i nazwanymi.
3. **Średnia ocen:** Napisz funkcję `oblicz_srednia(*args)`, która przyjmuje dowolną liczbę ocen (argumentów pozycyjnych) i zwraca ich średnią arytmetyczną. Jeśli nie podano żadnej oceny, powinna zwrócić 0.
4. **Sprawdzanie zakresu:** Zdefiniuj zmienną globalną `POZIOM_DOSTEPU = "user"`. Napisz funkcję, która próbuje zmienić tę zmienną na `"admin"` bez użycia słowa kluczowego `global`. Wewnątrz funkcji stwórz zmienną lokalną o tej samej nazwie. Wyświetl wartość zmiennej wewnątrz i na zewnątrz funkcji, aby zobaczyć różnicę.
5. **Adnotacje i docstring:** Weź funkcję `kalkulator` z zadania 1. Dodaj do niej pełne adnotacje typów dla wszystkich parametrów i wartości zwracanej. Napisz również kompletny docstring opisujący jej działanie.
6. **Wielokrotne powitanie:** Napisz funkcję `wielokrotne_powitanie(imie: str, ilosc: int) -> None`, która wyświetla powitanie `f"Cześć, {imie}!"` tyle razy, ile wynosi `ilosc`. Ta funkcja nie powinna niczego zwracać.
7. **Zwracanie wielu wartości:** Stwórz funkcję `analiza_listy(lista: list[int])`, która przyjmuje listę liczb i zwraca krotkę zawierającą trzy wartości: minimum, maksimum i sumę elementów z listy.
8. **Tworzenie profilu:** Napisz funkcję `stworz_profil(imie, **dane_dodatkowe)`, która przyjmuje imię oraz dowolną liczbę nazwanych argumentów (np. `wiek=30`, `miasto="Warszawa"`). Funkcja powinna zwrócić słownik z profilem użytkownika, gdzie klucz `'imie'` jest obowiązkowy, a reszta danych jest pobierana z `**dane_dodatkowe`.
9. **Silnia (rekurencja):** Napisz funkcję `silnia(n: int) -> int`, która oblicza silnię liczby `n` w sposób rekurencyjny (czyli wywołując samą siebie). Pamiętaj o warunku bazowym: silnia z 0 to 1. (Wzór: `n! = n * (n-1)!`).
10. **Mini-projekt: Walidator hasła:** Stwórz funkcję `sprawdz_haslo(haslo: str) -> bool`. Funkcja powinna sprawdzać, czy hasło spełnia następujące warunki i zwracać `True` lub `False`:
    - Ma co najmniej 8 znaków.
    - Zawiera co najmniej jedną wielką literę.
    - Zawiera co najmniej jedną cyfrę. Napisz do niej pełną dokumentację (docstring i adnotacje).